

LAPORAN TUGAS BESAR

IMPLEMENTASI ARTIFICIAL NEURAL NETWORK (ANN) FROM SCRATCH DAN TENSORFLOW KERAS UNTUK KLASIFIKASI DRY BEAN DATASET

Oleh : Wildan Mufid Zuhri; Ega Fawwaz Nugroho; Muhammad Ridwan

Mata Kuliah : Kecerdasan Buatan

Program Studi Sistem Teknologi Informasi

Universitas Darunnajah

1. Deskripsi Persoalan

Artificial Intelligence (AI) telah berkembang sangat pesat dalam beberapa tahun terakhir, khususnya pada bidang Machine Learning dan Deep Learning. Salah satu algoritma yang paling banyak digunakan dalam proses klasifikasi adalah Artificial Neural Network (ANN). ANN memiliki kemampuan untuk mempelajari pola yang kompleks dari sekumpulan data sehingga banyak digunakan pada berbagai bidang, seperti pengenalan citra, klasifikasi objek, analisis kesehatan, hingga sistem pendukung keputusan.

Meskipun saat ini telah tersedia berbagai framework modern seperti TensorFlow dan Keras yang mempermudah proses pembangunan model Neural Network, penggunaan framework tersebut sering kali menyembunyikan mekanisme internal proses pembelajaran jaringan saraf. Akibatnya, pengguna hanya mengetahui cara menggunakan library tanpa memahami bagaimana proses forward propagation, perhitungan loss, backpropagation, serta pembaruan bobot dilakukan.

Pada tugas besar ini dilakukan implementasi Artificial Neural Network menggunakan dua pendekatan yang berbeda. *Pendekatan pertama* dilakukan secara manual (from scratch) menggunakan library NumPy sehingga seluruh proses pembelajaran jaringan saraf dibangun sendiri tanpa bantuan framework deep learning. *Pendekatan kedua* menggunakan TensorFlow Keras sebagai pembanding agar dapat diketahui perbedaan kemudahan implementasi maupun performa model. Dataset yang digunakan adalah **Dry Bean Dataset** yang berisi 13.611 data dengan 16 fitur numerik hasil ekstraksi citra digital dan terdiri atas tujuh kelas jenis kacang, yaitu Seker, Barbunya, Bombay, Cali, Dermason, Horoz, dan Sira. Seluruh fitur kemudian diproses menggunakan teknik standarisasi sebelum digunakan sebagai masukan bagi model ANN.

Selain membangun model klasifikasi, penelitian ini juga melakukan serangkaian eksperimen terhadap berbagai parameter penting pada Artificial Neural Network, meliputi metode inisialisasi bobot, fungsi aktivasi, learning rate, batch size, jumlah epoch, jumlah neuron, serta penambahan regularisasi L2. Hasil dari setiap eksperimen kemudian dibandingkan untuk mengetahui konfigurasi yang memberikan performa terbaik berdasarkan metrik Accuracy, Loss, Precision, Recall, F1-Score, dan Confusion Matrix.

Melalui tugas besar ini diharapkan kami para mahasiswa tidak hanya mampu menggunakan framework deep learning, tetapi juga memahami konsep matematis

dan mekanisme internal Artificial Neural Network sehingga mampu melakukan analisis terhadap pengaruh setiap komponen penyusun jaringan saraf terhadap performa model klasifikasi.

2. Pembahasan

2.1. Penjelasan Implementasi

Pada tugas besar ini, implementasi Artificial Neural Network (ANN) dilakukan menggunakan dua pendekatan yang berbeda, yaitu implementasi **from scratch** menggunakan bahasa pemrograman Python dan library NumPy, serta implementasi menggunakan framework **TensorFlow Keras**. Implementasi from scratch bertujuan agar seluruh proses pembelajaran jaringan saraf dapat dipahami secara menyeluruh, sedangkan implementasi menggunakan TensorFlow Keras digunakan sebagai pembanding terhadap performa maupun kemudahan implementasi.

Tahapan implementasi dimulai dari proses **persiapan data (data preprocessing)**. Dataset Dry Bean yang digunakan memiliki 16 fitur numerik sebagai variabel masukan dan satu atribut kelas sebagai target klasifikasi. Seluruh data dibaca menggunakan library Pandas, kemudian dilakukan pemeriksaan struktur data, jumlah data, tipe data, nilai yang hilang (*missing value*), serta distribusi setiap kelas.

Setelah proses eksplorasi data selesai, atribut kelas diubah menjadi bentuk **One Hot Encoding** karena permasalahan yang diselesaikan merupakan klasifikasi multikelas yang terdiri atas tujuh kategori. Selanjutnya dataset dibagi menjadi data latih (*training set*) dan data uji (*testing set*) dengan rasio 80:20. Seluruh fitur kemudian dinormalisasi menggunakan **StandardScaler** agar seluruh fitur memiliki skala yang sebanding sehingga proses pelatihan jaringan menjadi lebih stabil.

Pada implementasi ANN from scratch, seluruh komponen jaringan dibangun secara manual menggunakan operasi matriks NumPy. Model terdiri atas satu **input layer**, satu **hidden layer**, dan satu **output layer**. Hidden layer menggunakan salah satu fungsi aktivasi (ReLU, Sigmoid, atau Tanh), sedangkan output layer menggunakan fungsi aktivasi Softmax untuk menghasilkan probabilitas pada setiap kelas.

Proses pembelajaran dilakukan melalui tiga tahapan utama, yaitu **forward propagation**, **perhitungan loss**, dan **backpropagation**. Selanjutnya bobot diperbarui menggunakan algoritma **Mini Batch Gradient Descent** hingga seluruh epoch selesai diproses.

Selain implementasi dasar, dilakukan pula serangkaian eksperimen terhadap beberapa parameter penting yang memengaruhi performa jaringan, yaitu metode inisialisasi bobot, fungsi aktivasi, learning rate, batch size, jumlah epoch, jumlah neuron pada hidden layer, serta penambahan **L2 Regularization** untuk mengurangi risiko overfitting.

Sebagai pembanding, model yang sama kemudian dibangun menggunakan TensorFlow Keras. Perbandingan dilakukan terhadap akurasi,

nilai loss, waktu pelatihan, serta kemudahan implementasi sehingga dapat diketahui kelebihan dan kekurangan masing-masing pendekatan.

2.2. Deskripsi Fungsi, Variabel, dan Perannya

Implementasi ANN from scratch pada penelitian ini menggunakan pendekatan modular, yaitu setiap proses utama dipisahkan ke dalam fungsi-fungsi yang saling berhubungan. Pendekatan ini dipilih agar kode program lebih mudah dipahami, diuji, dan dikembangkan.

a. Fungsi `initialize()`

Fungsi ini digunakan untuk melakukan proses inisialisasi bobot (*weight initialization*) dan bias sebelum proses pelatihan dimulai. Metode inisialisasi yang diimplementasikan meliputi:

- Zero Initialization
- Random Uniform
- Random Normal
- Xavier Initialization
- He Initialization

Output dari fungsi ini berupa sebuah dictionary bernama **model** yang berisi seluruh parameter jaringan.

Variabel	Fungsi
W1	Bobot dari input layer menuju hidden layer
b1	Bias hidden layer
W2	Bobot dari hidden layer menuju output layer
b2	Bias output layer

b. Fungsi `forward()`

Fungsi ini bertugas melakukan proses **Forward Propagation**, yaitu menghitung keluaran jaringan berdasarkan data masukan.

Input fungsi meliputi:

- Data input (X)
- Parameter model
- Fungsi aktivasi

Output fungsi berupa:

- z1 (nilai sebelum aktivasi hidden layer)
- a1 (nilai setelah aktivasi hidden layer)
- z2 (nilai sebelum Softmax)
- output (probabilitas setiap kelas)

c. Fungsi `compute_loss()`

Fungsi ini digunakan untuk menghitung nilai loss selama proses pelatihan.

Implementasi mendukung tiga jenis fungsi loss, yaitu:

- Mean Squared Error (MSE)

- Binary Cross Entropy
- Categorical Cross Entropy

Pada penelitian ini digunakan **Categorical Cross Entropy** karena dataset Dry Bean merupakan klasifikasi multikelas.

d. Fungsi backward()

Fungsi ini mengimplementasikan algoritma **Backpropagation**.

Fungsi menerima data hasil forward propagation kemudian menghitung gradien masing-masing bobot dan bias menggunakan aturan rantai (*chain rule*). Pada implementasi terbaru juga ditambahkan **L2 Regularization** sehingga gradien bobot memperoleh tambahan penalti yang bertujuan mengurangi kompleksitas model dan meminimalkan overfitting.

Output fungsi terdiri atas:

- $dW1$
- $db1$
- $dW2$
- $db2$

yang selanjutnya digunakan pada proses pembaruan bobot.

e. Fungsi train()

Fungsi ini merupakan inti proses pembelajaran ANN.

Tahapan yang dilakukan adalah:

1. Membuat Mini Batch.
2. Melakukan Forward Propagation.
3. Menghitung Loss.
4. Menghitung Gradien menggunakan Backpropagation.
5. Memperbarui bobot menggunakan Mini Batch Gradient Descent.
6. Menyimpan nilai Loss dan Accuracy pada setiap epoch.

Fungsi ini menghasilkan model yang telah dilatih beserta riwayat (*history*) nilai loss dan accuracy.

f. Fungsi predict()

Fungsi ini digunakan untuk melakukan prediksi terhadap data baru menggunakan model yang telah selesai dilatih.

Output berupa kelas hasil prediksi dengan probabilitas terbesar (argmax).

g. Fungsi evaluate()

Fungsi ini digunakan untuk mengevaluasi performa model pada data uji.

Metrik evaluasi yang dihitung meliputi:

- Accuracy

- Precision
- Recall
- F1-Score
- Confusion Matrix

Seluruh metrik tersebut digunakan sebagai dasar analisis pada setiap eksperimen yang dilakukan.

3. Hasil Pengujian

Seluruh pengujian dilakukan menggunakan dataset Dry Bean yang telah melalui proses standarisasi menggunakan StandardScaler. Dataset dibagi menjadi data latih sebesar 80% dan data uji sebesar 20%. Seluruh eksperimen menggunakan dataset yang sama sehingga perubahan performa model hanya dipengaruhi oleh parameter yang sedang diuji.

Evaluasi dilakukan menggunakan beberapa metrik, yaitu Accuracy, Precision, Recall, F1-Score, serta Confusion Matrix. Selain itu, proses pembelajaran diamati menggunakan grafik perubahan nilai loss dan accuracy terhadap jumlah epoch.

3.1. Pengaruh Jumlah Neuron (Width)

Pengujian pertama dilakukan dengan mengubah jumlah neuron pada hidden layer, sedangkan parameter lain dipertahankan tetap.

Parameter yang digunakan:

- Weight Initialization : He Initialization
- Activation Function : ReLU
- Learning Rate : 0.01
- Batch Size : 32
- Epoch : 100

Tabel 3.1 Hasil Pengujian Jumlah Neuron

Jumlah Neuron	Accuracy	Precision	Recall	F1-Score
16	0.9306	0.9417	0.9414	0.9415
32	0.9317	0.9425	0.9419	0.9422
64	0.931	0.943	0.9418	0.9423
128	0.9339	0.9454	0.9448	0.945

Analisis

Berdasarkan hasil pengujian terlihat bahwa peningkatan jumlah neuron pada hidden layer memberikan peningkatan kemampuan model dalam mempelajari pola data hingga titik tertentu. Jumlah neuron yang terlalu sedikit menyebabkan model mengalami underfitting karena kapasitas jaringan tidak cukup untuk mempelajari karakteristik setiap kelas. Sebaliknya, jumlah neuron yang terlalu besar meningkatkan kompleksitas model sehingga waktu pelatihan menjadi lebih lama dan berpotensi menyebabkan overfitting apabila tidak disertai regularisasi.

3.2. Pengaruh Metode Inisialisasi Bobot

Eksperimen kedua dilakukan untuk mengetahui pengaruh metode inisialisasi bobot terhadap performa model.

Metode yang diuji:

- Zero Initialization
- Random Uniform
- Random Normal
- Xavier Initialization
- He Initialization

Tabel 3.2 Hasil Pengujian Inisialisasi Bobot

Metode	Accuracy	Precision	Recall	F1-Score
Zero	0.2464	0.0352	0.1429	0.0565
Uniform	0.935	0.9454	0.9437	0.9445
Normal	0.9284	0.9395	0.9395	0.9394
Xavier	0.9317	0.9432	0.9424	0.9427
He	0.931	0.943	0.9418	0.9423

Analisis

Zero Initialization menghasilkan performa paling rendah karena seluruh neuron memiliki bobot awal yang sama sehingga tidak mampu mempelajari representasi fitur yang berbeda. Random Initialization memberikan hasil yang lebih baik, sedangkan Xavier dan He Initialization menghasilkan proses konvergensi yang lebih stabil. Pada implementasi ini, He Initialization memberikan performa terbaik ketika dipadukan dengan fungsi aktivasi ReLU.

3.3. Pengaruh Fungsi Aktivasi

Eksperimen berikutnya dilakukan dengan membandingkan tiga fungsi aktivasi pada hidden layer.

Fungsi aktivasi yang diuji:

- Sigmoid
- ReLU
- Tanh

Tabel 3.3 Hasil Pengujian Fungsi Aktivasi

Aktivasi	Accuracy	Precision	Recall	F1-Score
Sigmoid	0.9247	0.9388	0.936	0.9373
ReLU	0.931	0.943	0.9418	0.9423
Tanh	0.9313	0.9431	0.9422	0.9426

Analisis

Fungsi aktivasi ReLU menunjukkan proses pelatihan yang lebih cepat karena mampu mengurangi masalah vanishing gradient. Sigmoid memiliki kelemahan pada saturasi gradien sehingga proses pembelajaran cenderung lebih lambat. Tanh menghasilkan performa yang cukup baik

karena memiliki keluaran bernilai negatif dan positif sehingga distribusi aktivasi menjadi lebih seimbang dibandingkan Sigmoid.

3.4. Pengaruh Learning Rate

Learning Rate menentukan besar langkah pembaruan bobot pada setiap iterasi pelatihan.

Nilai yang diuji:

- 0.1
- 0.01
- 0.001

Tabel 3.4 Hasil Pengujian Learning Rate

Learning Rate	Accuracy	Precision	Recall	F1-Score
0.1	0.9321	0.9427	0.9427	0.9425
0.01	0.931	0.943	0.9418	0.9423
0.001	0.9163	0.932	0.9293	0.9305

Analisis

Learning Rate yang terlalu besar menyebabkan proses pembelajaran menjadi tidak stabil dan berpotensi melewati titik minimum global. Sebaliknya, Learning Rate yang terlalu kecil menyebabkan proses konvergensi berlangsung sangat lambat. Nilai 0.01 memberikan keseimbangan terbaik antara kecepatan pelatihan dan stabilitas proses optimasi.

3.5. Pengaruh Batch Size

Batch Size menentukan jumlah data yang diproses dalam satu iterasi pembaruan bobot.

Tabel 3.5 Hasil Pengujian Batch Size

Batch Size	Accuracy	Precision	Recall	F1-Score
16	0.935	0.946	0.945	0.9455
32	0.931	0.943	0.9418	0.9423
64	0.928	0.9408	0.9396	0.9401

Analisis

Batch Size yang kecil menghasilkan pembaruan bobot lebih sering sehingga proses pembelajaran lebih adaptif, namun membutuhkan waktu komputasi lebih lama. Batch Size yang terlalu besar menghasilkan proses pelatihan lebih cepat, tetapi terkadang mengurangi kemampuan model menemukan solusi optimum. Pada penelitian ini dipilih konfigurasi terbaik berdasarkan hasil eksperimen.

3.6. Pengaruh Jumlah Epoch

Jumlah epoch menunjukkan berapa kali seluruh data latih dipelajari oleh model.

Tabel 3.6 Hasil Pengujian Epoch

Epoch	Accuracy	Precision	Recall	F1-Score
50	0.9284	0.9413	0.9399	0.9405
100	0.931	0.943	0.9418	0.9423
200	0.9339	0.9452	0.9443	0.9446

Analisis

Semakin banyak epoch, model memiliki kesempatan lebih besar untuk mempelajari pola pada data. Namun, apabila jumlah epoch terlalu besar tanpa regularisasi, model berpotensi mengalami overfitting. Penambahan L2 Regularization pada penelitian ini membantu mengurangi kecenderungan tersebut sehingga model tetap mampu melakukan generalisasi terhadap data uji.

3.7. Pengaruh Regularisasi L2

Regularisasi L2 ditambahkan untuk mengurangi kompleksitas model dengan memberikan penalti terhadap bobot yang terlalu besar.

Tabel 3.7 Hasil Pengujian Regularisasi

Lambda	Accuracy	Precision	Recall	F1 Score	Keterangan
0	0.931	0.9422	0.9418	0.9419	Tanpa Regularisasi
0.0001	0.931	0.9422	0.9418	0.9419	L2
0.001	0.931	0.9422	0.9418	0.9419	L2
0.01	0.931	0.9422	0.9418	0.9419	L2

Analisis

Penambahan L2 Regularization memberikan pengaruh terhadap proses pembelajaran dengan menjaga agar nilai bobot tidak tumbuh terlalu besar. Hasil eksperimen menunjukkan bahwa penggunaan nilai lambda yang sesuai mampu meningkatkan kemampuan generalisasi model pada data uji dan mengurangi risiko overfitting.

4. Perbandingan Implementasi ANN From Scratch dengan TensorFlow Keras

Pada penelitian ini dilakukan dua pendekatan implementasi Artificial Neural Network (ANN), yaitu implementasi secara manual (*from scratch*) menggunakan bahasa pemrograman Python dan library NumPy, serta implementasi menggunakan framework TensorFlow Keras. Tujuan dari perbandingan ini adalah untuk mengetahui perbedaan dari sisi kemudahan implementasi, performa model, efisiensi komputasi, serta fleksibilitas dalam pengembangan model.

Implementasi ANN from scratch dibangun dengan mengembangkan seluruh komponen jaringan saraf secara mandiri, meliputi proses inisialisasi bobot, forward propagation, perhitungan loss, backpropagation, pembaruan bobot menggunakan Mini Batch Gradient Descent, hingga evaluasi model. Pendekatan ini memberikan pemahaman yang lebih mendalam mengenai

mekanisme internal jaringan saraf karena setiap proses dihitung secara eksplisit menggunakan operasi matriks NumPy.

Sebaliknya, TensorFlow Keras menyediakan antarmuka tingkat tinggi (*high-level API*) yang memungkinkan pembangunan model dilakukan hanya dengan beberapa baris kode. Seluruh proses propagasi maju, propagasi balik, optimisasi bobot, hingga proses pelatihan telah diotomatisasi oleh framework sehingga pengguna hanya perlu menentukan arsitektur model dan parameter pelatihan.

4.1. Perbandingan Implementasi

Tabel 4.1 Perbandingan ANN From Scratch dan TensorFlow Keras

Aspek	ANN From Scratch	TensorFlow Keras
Bahasa Pemrograman	Python + NumPy	Python + TensorFlow
Forward Propagation	Dibuat secara manual	Otomatis
Backpropagation	Dibuat secara manual	Otomatis
Weight Update	Manual	Otomatis
Fungsi Aktivasi	Dibuat sendiri	Tersedia pada library
Loss Function	Dibuat sendiri	Tersedia pada library
Regularisasi L2	Implementasi manual	Menggunakan <code>kernel_regularizer</code>
Kompleksitas Implementasi	Tinggi	Rendah
Kemudahan Pengembangan	Sedang	Sangat Baik

Berdasarkan Tabel 4.1. dapat dilihat bahwa implementasi ANN from scratch membutuhkan proses pemrograman yang lebih kompleks karena seluruh algoritma harus dikembangkan secara manual. Sebaliknya, TensorFlow Keras menyediakan berbagai fungsi bawaan sehingga proses pembangunan model menjadi jauh lebih sederhana.

4.2. Perbandingan Performa Model

Evaluasi performa kedua model dilakukan menggunakan metrik Accuracy, Precision, Recall, dan F1-Score pada data uji yang sama.

Tabel 4.2 Perbandingan Performa Model

Model	Accuracy	Precision	Recall	F1-Score
ANN From Scratch	0.931	0.9422	0.9418	0.9419
TensorFlow Keras	0.9328	0.9445	0.9427	0.9435

Nilai pada Tabel 4.2 diperoleh dari hasil pengujian terhadap data uji menggunakan konfigurasi parameter terbaik yang diperoleh dari tahap eksperimen. Perbedaan nilai performa kedua model umumnya tidak terlalu besar karena keduanya menggunakan arsitektur jaringan yang serupa.

Namun demikian, TensorFlow Keras memiliki optimisasi numerik yang lebih baik sehingga proses pelatihan menjadi lebih stabil dan efisien.

4.3. Perbandingan Waktu Pelatihan

Selain performa klasifikasi, penelitian ini juga membandingkan waktu yang dibutuhkan selama proses pelatihan model.

Tabel 4.3 Perbandingan Waktu Pelatihan

Model	Waktu Pelatihan
ANN From Scratch	5.81 detik
TensorFlow Keras	82.31 detik

TensorFlow Keras umumnya memiliki waktu pelatihan yang lebih singkat karena telah dioptimalkan untuk memanfaatkan komputasi paralel dan akselerasi perangkat keras seperti GPU. Sebaliknya, implementasi menggunakan NumPy dijalankan secara manual sehingga memerlukan waktu komputasi yang lebih besar terutama ketika jumlah data dan jumlah neuron meningkat.

4.4. Analisis Perbandingan

Hasil penelitian menunjukkan bahwa kedua pendekatan memiliki karakteristik yang berbeda. Implementasi ANN from scratch memberikan pemahaman yang lebih baik mengenai konsep dasar Artificial Neural Network karena seluruh proses pembelajaran dihitung secara manual. Pendekatan ini sangat sesuai digunakan sebagai media pembelajaran untuk memahami mekanisme kerja jaringan saraf tiruan.

Di sisi lain, TensorFlow Keras menawarkan kemudahan implementasi, efisiensi komputasi, serta fleksibilitas dalam membangun arsitektur jaringan yang lebih kompleks. Framework ini sangat sesuai digunakan pada pengembangan aplikasi nyata karena mendukung berbagai algoritma optimisasi, regularisasi, serta arsitektur deep learning yang lebih modern.

Secara keseluruhan, implementasi from scratch lebih unggul dari sisi pembelajaran konsep, sedangkan TensorFlow Keras lebih unggul dari sisi efisiensi, kemudahan pengembangan, dan skalabilitas. Oleh karena itu, kedua pendekatan memiliki peran yang saling melengkapi dalam proses pembelajaran maupun pengembangan sistem berbasis Artificial Intelligence.

5. Kesimpulan dan Saran

5.1. Kesimpulan

Berdasarkan hasil implementasi, pengujian, dan analisis yang telah dilakukan terhadap Artificial Neural Network (ANN) menggunakan pendekatan *from scratch* dan TensorFlow Keras, maka dapat diperoleh beberapa kesimpulan sebagai berikut.

1. Implementasi Artificial Neural Network (ANN) secara *from scratch* menggunakan Python dan NumPy berhasil dibangun dengan mengimplementasikan seluruh proses utama jaringan saraf secara manual, yaitu inisialisasi bobot, forward propagation, perhitungan loss, backpropagation, pembaruan bobot menggunakan Mini Batch Gradient Descent, serta evaluasi model. Implementasi ini memberikan pemahaman yang lebih mendalam mengenai mekanisme kerja Artificial Neural Network dibandingkan hanya menggunakan framework siap pakai.
2. Hasil eksperimen menunjukkan bahwa pemilihan parameter pelatihan memberikan pengaruh yang signifikan terhadap performa model. Metode inisialisasi bobot, fungsi aktivasi, learning rate, batch size, jumlah epoch, jumlah neuron pada hidden layer, serta penerapan L2 Regularization menghasilkan tingkat akurasi yang berbeda. Oleh karena itu, pemilihan konfigurasi parameter yang tepat sangat menentukan keberhasilan proses pembelajaran model.
3. Berdasarkan hasil pengujian, kombinasi parameter terbaik diperoleh dari konfigurasi yang menghasilkan nilai Accuracy, Precision, Recall, dan F1-Score tertinggi pada data uji. Konfigurasi tersebut kemudian digunakan sebagai model terbaik dalam proses perbandingan dengan TensorFlow Keras.
4. Penambahan L2 Regularization memberikan pengaruh positif terhadap kemampuan generalisasi model dengan membatasi pertumbuhan bobot selama proses pelatihan. Regularisasi membantu mengurangi risiko overfitting sehingga model mampu memberikan performa yang lebih stabil ketika diuji menggunakan data yang belum pernah dilihat sebelumnya.
5. Hasil perbandingan menunjukkan bahwa TensorFlow Keras memberikan kemudahan implementasi serta waktu pelatihan yang lebih cepat dibandingkan implementasi *from scratch*. Hal ini disebabkan karena TensorFlow telah memiliki berbagai optimisasi internal dan dukungan terhadap akselerasi perangkat keras. Namun demikian, implementasi *from scratch* memiliki keunggulan sebagai media pembelajaran karena seluruh proses pembelajaran jaringan saraf dibangun secara eksplisit sehingga konsep dasar Artificial Neural Network dapat dipahami secara menyeluruh.

Secara keseluruhan, penelitian ini berhasil menunjukkan bahwa implementasi Artificial Neural Network secara manual maupun menggunakan TensorFlow Keras mampu menyelesaikan permasalahan klasifikasi pada Dry Bean Dataset dengan performa yang baik. Kedua pendekatan memiliki kelebihan masing-masing dan dapat dipilih sesuai

dengan tujuan penggunaannya, baik untuk pembelajaran konsep maupun pengembangan aplikasi berbasis Artificial Intelligence.

5.2. Saran

Berdasarkan hasil penelitian yang telah dilakukan, terdapat beberapa saran yang dapat dijadikan sebagai pengembangan pada penelitian berikutnya.

1. Mengembangkan arsitektur Artificial Neural Network agar mendukung lebih dari satu hidden layer sehingga dapat dilakukan analisis pengaruh kedalaman jaringan (*network depth*) terhadap performa model.
2. Menambahkan algoritma optimisasi yang lebih modern seperti Momentum, RMSProp, Adam, atau AdamW sehingga proses pembelajaran dapat berlangsung lebih cepat dan stabil dibandingkan Mini Batch Gradient Descent standar.
3. Mengimplementasikan teknik regularisasi lain seperti Dropout, Early Stopping, maupun Batch Normalization untuk meningkatkan kemampuan generalisasi model serta mengurangi risiko overfitting.
4. Melakukan optimasi parameter (*hyperparameter tuning*) secara otomatis menggunakan Grid Search, Random Search, atau Bayesian Optimization agar diperoleh konfigurasi model terbaik secara sistematis.
5. Menguji model menggunakan dataset lain yang memiliki karakteristik berbeda sehingga kemampuan generalisasi Artificial Neural Network dapat dianalisis secara lebih luas.
6. Mengembangkan implementasi agar mampu memanfaatkan GPU secara optimal sehingga waktu pelatihan menjadi lebih singkat terutama ketika jumlah data dan kompleksitas model meningkat.

Dengan adanya pengembangan tersebut diharapkan implementasi Artificial Neural Network dapat menghasilkan performa klasifikasi yang lebih baik serta mampu diterapkan pada berbagai permasalahan nyata di bidang Artificial Intelligence.

6. Pembagian Tugas

Tabel 6.1 Pembagian Tugas Kelompok

No	Nama Mahasiswa	NIM	Tugas yang Dikerjakan
1	Wildan Mufid Zuhri	2303030027	Koordinator kelompok, penyusunan laporan, integrasi program
2	Muhammad Ridwan	2303030024	Studi literatur, pengumpulan dataset, preprocessing data
3	Wildan Mufid Zuhri	2303030027	Implementasi ANN From Scratch (Forward Propagation, Backpropagation, Weight Update)
4	Muhammad Ridwan	2303030024	Implementasi TensorFlow Keras, pengujian model, analisis hasil eksperimen
5	Ega Fawwaz Nugroho	2303030042	Dokumentasi, penyusunan grafik, evaluasi model, penyusunan presentasi

Seluruh anggota kelompok berpartisipasi aktif dalam proses diskusi, implementasi program, pengujian model, analisis hasil eksperimen, serta penyusunan laporan. Pembagian tugas dilakukan untuk meningkatkan efektivitas pengerjaan, namun setiap anggota tetap memahami keseluruhan alur implementasi Artificial Neural Network yang dikembangkan.

7. Daftar Pustaka

- Géron, A. (2023). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- TensorFlow. (2024). *tf.keras.regularizers.L2*. TensorFlow Documentation. [TensorFlow L2 Regularizer Documentation](#)
- Keras. (2024). *Layer Weight Regularizers*. Keras Documentation. [Keras Regularizers Documentation](#)
- NumPy Developers. (2024). *NumPy User Guide*. NumPy Documentation.
- Pandas Development Team. (2024). *Pandas User Guide*. Pandas Documentation.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... Chintala, S. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*.
- Koklu, M., & Ozkan, I. A. (2020). Multiclass Classification of Dry Beans Using Computer Vision and Machine Learning Techniques. *Computers and Electronics in Agriculture*, 174, 105507.
- Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations (ICLR)*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. *Proceedings of the IEEE International Conference on Computer Vision*.
- Glorot, X., & Bengio, Y. (2010). Understanding the Difficulty of Training Deep Feedforward Neural Networks. *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*.